Extra Time - Cahier des charges

Ce document précise le cadre technique et le contrat d'intégration. Les règles métier, les cas limites et les comportements attendus doivent provenir de l'entretien avec l'expert métier.

1. Travail attendu

L'équipe réalise une application back-end qui :

- expose une API REST ;
- conserve durablement les informations ;
- applique les règles recueillies pendant l'entretien ;
- peut être démarrée et testée de manière reproductible ;
- documente son installation, son exécution et ses principales décisions ;
- sépare les responsabilités de présentation, d'application, de domaine et de persistance.

Le modèle du domaine et l'organisation interne du code sont conçus par l'équipe.

2. Stack et environnement d'exécution

La stack technologique est libre, sous réserve de validation avant le développement.

La stack de référence utilise TypeScript, Bun, Hono, Zod, SQLite, `bun:test`, et Bruno. Le kit de départ est prévu pour cette stack, mais son utilisation est facultative.

L'API est accessible sur `http://localhost:3000`. Le port est configurable avec la variable d'environnement `PORT`.

L'application expose `GET /health`. Lorsque le service est prêt, cette route répond :

```
200 OK
Content-Type: application/json

{
  "status": "ok"
}
```

Le dépôt documente les commandes suivantes ou leur équivalent :

Action	Stack de référence
Démarrer l'API	`bun run start`
Initialiser ou migrer le stockage	`bun run db:migrate`
Lancer les tests	`bun test`
Lancer le parcours API	`bun run test:api`

Les dépendances sont déclarées et verrouillées. Les variables nécessaires sont décrites dans `.env.example`, sans secret.

3. Éléments fournis

Le kit de départ contient :

- un composant SSE (facultatif) ;
- un pipeline d'intégration continue (facultatif) ;
- un `Dockerfile` et un fichier `compose.yaml` ;
- une collection Bruno de départ ;
- une page de démonstration abonnée au flux SSE ;
- un exemple de configuration.

Le démarrage de l'application, le stockage, son initialisation, les routes REST et la logique métier ne sont pas fournis.

La conteneurisation et le pipeline sont des moyens d'exécution. Leur conception n'est pas évaluée.

4. Contrat d'intégration

Le contrat ci-dessous définit les routes obligatoires de création, de consultation et d'enregistrement. La route SSE de la section 4.5 est facultative. Ce contrat définit une représentation publique, pas la structure interne de l'application.

4.1 Représentation publique

```
{
  "id": "opaque-id",
  "homeTeam": "France",
  "awayTeam": "Italie",
  "startingTeam": "HOME",
  "status": "IN_PROGRESS",
  "winner": null,
  "score": {
    "home": 0,
    "away": 0
  },
  "nextTeam": "HOME",
  "version": 0,
  "createdAt": "2026-10-17T12:00:00.000Z",
  "shots": []
}
```

Un élément de `shots` respecte la forme suivante :

```
{
  "sequence": 1,
  "team": "HOME",
  "shooter": "Griezmann",
  "goalkeeper": "Donnarumma",
  "outcome": "GOAL",
  "clientShotId": "client-shot-001",
  "recordedAt": "2026-10-17T12:00:05.000Z"
}
```

Les identifiants et les dates sont produits par le serveur. Les dates sont sérialisées au format ISO 8601 en UTC.

Les champs `shooter` et `goalkeeper` contiennent le nom du tireur et celui du gardien adverse, de 1 à 60 caractères. Les valeurs de `startingTeam`, `team` et `nextTeam` sont `HOME` ou `AWAY`. Le champ `status` vaut `IN_PROGRESS` ou `FINISHED`. Lorsque la séance est en cours, `winner` vaut `null` et `nextTeam` désigne l'équipe attendue. Lorsque la séance est terminée, `winner` désigne l'équipe gagnante et `nextTeam` vaut `null`.

4.2 Créer une ressource

`POST /shootouts`

```
{
  "homeTeam": "France",
  "awayTeam": "Italie",
  "startingTeam": "HOME"
}
```

Réponses possibles :

- `201 Created` avec la représentation publique et un en-tête `Location` ;
- `400 Bad Request` si la requête est invalide.

4.3 Consulter une ressource

`GET /shootouts/:id`

Réponses possibles :

- `200 OK` avec la représentation publique ;
- `404 Not Found` si la ressource n'existe pas.

4.4 Enregistrer une information

`POST /shootouts/:id/shots`

```
{
  "team": "HOME",
  "shooter": "Griezmann",
  "goalkeeper": "Donnarumma",
  "outcome": "GOAL",
  "clientShotId": "client-shot-001",
  "expectedVersion": 0
}
```

Réponses possibles :

- `201 Created` avec la représentation publique actualisée ;
- `400 Bad Request` si la requête est invalide ;
- `404 Not Found` si la ressource n'existe pas ;
- `409 Conflict` si la demande est incompatible avec l'état courant.

Les valeurs acceptées, les transitions autorisées et les différents motifs de conflit sont déterminés à partir des informations recueillies pendant l'entretien.

4.5 Suivre les mises à jour - facultatif

`GET /shootouts/:id/events`

Cette route peut être raccordée au composant SSE fourni. Le transport et la reconnexion ne sont pas à développer. L'absence de raccordement SSE n'est pas pénalisée.

Réponses possibles :

- `200 OK` avec un flux `text/event-stream` ;
- `404 Not Found` si la ressource n'existe pas.

À l'ouverture, le flux transmet l'état courant. Une modification acceptée et persistée produit ensuite un événement :

Champ SSE	Valeur
`event`	`shootout.updated`
`id`	version de la ressource après modification
`data`	représentation publique complète au format JSON

Les tests du transport SSE ne sont pas obligatoires.

5. Format des erreurs

Une erreur utilise la structure suivante :

```
{
  "error": {
    "code": "CODE_STABLE",
    "message": "Description lisible de l'erreur."
  }
}
```

Le code doit être stable et exploitable par un client. Le message est destiné à la lecture d'un utilisateur.

Les codes associés aux refus métier et leur priorité sont consignés dans le registre des décisions à l'issue de l'entretien. Ils ne doivent pas être déduits du code fourni ou inventés pendant l'implémentation.

6. Contraintes techniques

- Une modification acceptée est persistée de manière atomique.
- Une demande refusée ne laisse aucune modification partielle.
- Le champ `expectedVersion` permet de détecter une écriture fondée sur un état qui n'est plus courant.
- Deux écritures fondées sur la même version ne doivent pas être acceptées silencieusement.
- Si le composant SSE est raccordé, un événement est publié uniquement après la persistance de la modification correspondante.
- Les tests disposent d'un stockage isolé ou réinitialisé.

- Les valeurs variables comme l'heure et les identifiants doivent pouvoir être maîtrisées dans les tests lorsque cela est nécessaire.

7. Vérifications minimales

La suite de tests comprend :

- des tests unitaires de la logique métier ;
- au moins un test fondé sur une propriété ;
- des tests d'intégration ciblés avec le stockage réel ;
- un parcours REST automatisé comportant une création, une consultation, une modification acceptée et une demande refusée.

Chaque test est relié à une exigence, une décision ou un risque identifié. Le nombre de tests n'est pas un objectif en soi.

8. Hors périmètre obligatoire

- développement des interfaces graphiques ;
- raccordement et tests du flux SSE ;
- tests de charge ;
- concurrence réellement parallèle ;
- authentification réelle ;
- déploiement ;
- tests de pénétration ;
- conception de la conteneurisation et du pipeline CI.

Toute règle ou évolution non confirmée pendant l'entretien reste hors du périmètre évalué.